Склеивание цепочек в MeshA*

Проверка ускорения, 29 октября 2026 года

Новая реализация сократила CPU-время на 16,4% на основном наборе и на 9,1% на отложенных картах. На задачах основного набора, где путь найден, сокращение составило 7,4%. Выигрыш есть, но его величина зависит от задач и нагрузки на Mac. С компактным CLOSED (следствие леммы 1, см. дополнение) склейка на Linux сокращает время на 20,4% и 18,4%.

Сравнение выполнено с исходной C++-реализацией MeshA*. Сборки работают с весом эвристики $w = 1$, точным финишем, `C++17 -O3` и штатными runtime-проверками. В обеих версиях используется исходный CLOSED. Изменение с исключением internal из CLOSED из предыдущего опыта в это сравнение не включено.

Что здесь склеивается. Примитив задаёт короткое движение робота. Initial-вершины соответствуют началам и концам примитивов, internal описывают промежуточные состояния. Если у internal единственное продолжение, последовательность таких состояний можно обработать как один переход. Все клетки цепочки по-прежнему проверяются на столкновения.

Вариант	Реализация	Зачем проверялся
Исходная	Общая очередь OPEN и действующий авторский прыжок к единственному допустимому завершению примитива.	База сравнения. Авторская оптимизация остаётся включённой.
Прежняя склейка	После извлечения узла проходит по единственному продолжению. На каждой клетке повторяет проверку pruning.	Воспроизведение первого прототипа и его накладных расходов.
Нулевые цепочки	До выделения узла и вставки в OPEN проходит заранее рассчитанную цепочку нулевой стоимости до внутренней развилки.	Убирает узлы очереди, повторные расчёты эвристики и проходы по Finals.
Полная склейка	При $w = 1$ также завершает однозначный примитив до initial и добавляет его полную стоимость к g.	Этот вариант выбран для основной серии до её запуска. При $w \neq 1$ используется только нулевая склейка.

Таблицы цепочек строятся один раз для загруженного набора примитивов. Проверяется единственность internal-предшественника, достижимость из initial и совпадение Finals после сдвига клетки. Если структура не подходит, склеивание отключается. Стоимость подготовки записывается отдельно и не входит во время поиска.

Почему сохраняется стоимость пути. Внутри цепочки нет выбора, а initial не пропускаются. Нулевые переходы сохраняют g и множество допустимых завершений. Если цепочка заканчивается единственным примитивом стоимости c, то при $w = 1$

$$f_{\text{internal}} = g + c + h(v) = (g + c) + h(v) = f_{\text{initial}}.$$

Поэтому промежуточные узлы не нужны для выбора меньшего приоритета. При вычислениях с `float` и равных приоритетах может измениться выбранный путь. Его стоимость и корректность проверяются отдельно. Идея corridor skipping уже описана в комментариях авторов. Здесь исследуется её реализация и измеряемый эффект.

Основная серия: 960 измеряемых поисков

В наборе 24 заранее выбранные задачи на шести картах, с наборами примитивов `base` и `add`. Выбор сделан по фиксированным диапазонам расстояния, с сохранённым seed. Задачи без пути оставлены. Во время основной серии сборка и другие тесты не запускались.

В каждом процессе сначала выполняется один неучитываемый круг прогрева, затем пять измеряемых кругов. Первый сеанс использует порядок ABBA, второй BAAB, где A означает исходную версию, B означает склеивание. Итого на задачу приходится 20 измерений каждого варианта. Счётчики и таймеры отдельных операций в этих бинарниках отсутствуют.

Основной показатель рассчитывается так: для каждой задачи берётся медиана CPU-времени её повторов, затем медианы суммируются. Время загрузки данных, создания CLOSED, восстановления пути и очистки в `AstarSearch` не входит. Все числа ниже даны в миллисекундах, ускорение равно отношению исходного времени к новому.

Группа	Задач	Исходная	Склейка	Ускорение	Меньше CPU
Все задачи	24	14 933,25	12 485,44	1,196×	16,4%
Путь найден	21	3 925,06	3 633,76	1,080×	7,4%
Пути нет	3	11 008,19	8 851,68	1,244×	19,6%

Три задачи без пути занимают около 74% исходного времени этого набора. Поэтому общий результат нельзя переносить на одни только успешные поиски. Геометрическое среднее ускорений по всем 24 задачам составляет 1,154 ×.

Карта	Задач	Исходная	Склейка	Ускорение
AR0304SR	3	683,30	642,26	1,064×
Berlin_2_256	5	4 320,66	4 171,06	1,036×
Labyrinth	3	759,27	697,35	1,089×
Milan_1_256	5	429,84	386,82	1,111×
Moscow_0_512	5	8 308,17	6 182,87	1,344×
WheelofWar	3	432,01	405,09	1,066×

Сеанс	Порядок	Исходная	Склейка	Меньше CPU
1	ABBA	14 759,75	11 894,65	19,4%
2	BAAB	15 335,74	13 980,50	8,8%

Направление эффекта повторилось в обоих сеансах, но величина изменилась. При bootstrap по шести картам, 10 000 выборок, описательный 95%-интервал общего ускорения составляет 1,04–1,32 ×. Карт мало, это не гарантия выигрыша на произвольной карте. Если добавить создание CLOSED и очистку, без восстановления и проверки пути, ускорение на основном наборе составляет 1,191 ×.

Отложенные карты и расходы алгоритма

После основной серии выполнены по три повтора всех четырёх вариантов на 24 задачах с четырёх других карт. Эти карты не использовались для изменения алгоритма. Дополнительная серия выполнялась отдельными блоками по вариантам, без ABBA, поэтому её проценты менее защищены от дрейфа нагрузки, чем основная серия.

Карта	Исходная	Прежняя	Нулевая	Полная
AR0307SR	1 375,73	1 152,12	1 061,99	983,41
AR0700SR	1 696,35	1 475,53	1 279,60	1 354,61
ht_0_hightown	6 035,80	6 027,33	5 890,42	6 011,61
w_woundedcoast	3 123,77	2 952,96	2 929,07	2 766,08
Все карты	12 231,65	11 607,95	11 161,08	11 115,72

На этом наборе прежний прототип сократил время примерно на 5,1%, нулевая склейка на 8,8%, полная на 9,1%. Разница между двумя новыми вариантами мала. На `AR0700SR` и `ht_0_hightown` нулевая склейка оказалась быстрее полной. Дополнительный шаг до initial не даёт устойчивого выигрыша на каждой карте.

Откуда берётся экономия. Для основного набора сделан отдельный проход с инструментированными бинарниками. В таблице суммы операций в миллионах за один круг из 24 задач. Время этих прогонов в оценку ускорения не входит.

Операции, млн	Исходная	Прежняя	Нулевая	Полная
Вставки в OPEN	24,114	20,219	20,124	18,142
Извлечения из OPEN	23,503	19,577	19,484	17,502
Расчёты эвристики	24,114	20,219	20,124	18,142
Вызовы pruning	23,962	34,764	20,795	20,766
Просмотр элементов Finals	236,960	211,550	188,946	182,831
Все проверки клеток	286,480	264,938	244,805	237,828
Удаление дубликатов	1,206	1,287	1,286	2,325

Полная склейка убрала 24,8% вставок в OPEN, 25,5% извлечений и 24,8% расчётов эвристики. Прежний прототип экономил на очереди, но чаще вызывал pruning: число вызовов выросло с 23,96 до 34,76 млн. В новом варианте такие проверки не повторяются на каждой промежуточной клетке.

У полной склейки есть цена: initial-состояния создаются раньше, и число удаляемых дубликатов выросло с 1,21 до 2,33 млн. Несмотря на это, общее число вставок и извлечений уменьшилось. По этим данным нельзя утверждать, что максимальное сжатие всегда выгоднее нулевого.

Проверка результатов. В основной серии и на отложенных картах совпали признак нахождения пути и стоимость. Между исходной и новой версиями на четырёх задачах основного набора выбран другой путь той же стоимости. Внутри каждого варианта повторы дают одинаковые хеши путей, числа раскрытий и проверок клеток.

Для найденных путей проверены начало и финиш, все примитивы, их collision traces и сумма стоимости. При сравнении стоимости используется допуск $\max(0{,}002, 10^{-5}C)$, где C означает сумму стоимостей примитивов исходного пути. Дополнительно пройдены 20 ручных случаев с пустой картой, обходом стены, недостижимым финишем и совпадением старта с целью, а также тесты структуры макропутей.

Дополнение: теория, компактный CLOSED и проверка на Linux

Почему склейка корректна. Обозначим конфигурацию Ψ как набор пар (примитив p, номер его клетки k). Все пары одной конфигурации имеют одинаковый k, а их примитивы совпадают по первым $k+1$ клеткам следа. Конфигурация однозначно задаётся курсом θ и этим общим префиксом.

Лемма 1 (единственный предок). У любой internal-клетки ровно один предок: сдвиг к нему равен шагу следа любого примитива из Ψ, а конфигурация предка состоит из всех незакончившихся примитивов пучка с префиксом на одну клетку короче. Поэтому развёртка каждого пучка — дерево, и internal-клетка порождается не более одного раза.

Лемма 2 (f постоянно на цепочке). Если у internal-клетки u единственный потомок w и он internal, то ни один примитив не заканчивается и все делают одинаковый шаг. Значит, в w те же примитивы, а множество концов $\mathrm{Finals}(w) = \mathrm{Finals}(u)$. Отсюда $g(w) = g(u)$, $h(w) = h(u)$, $f(w) = f(u)$.

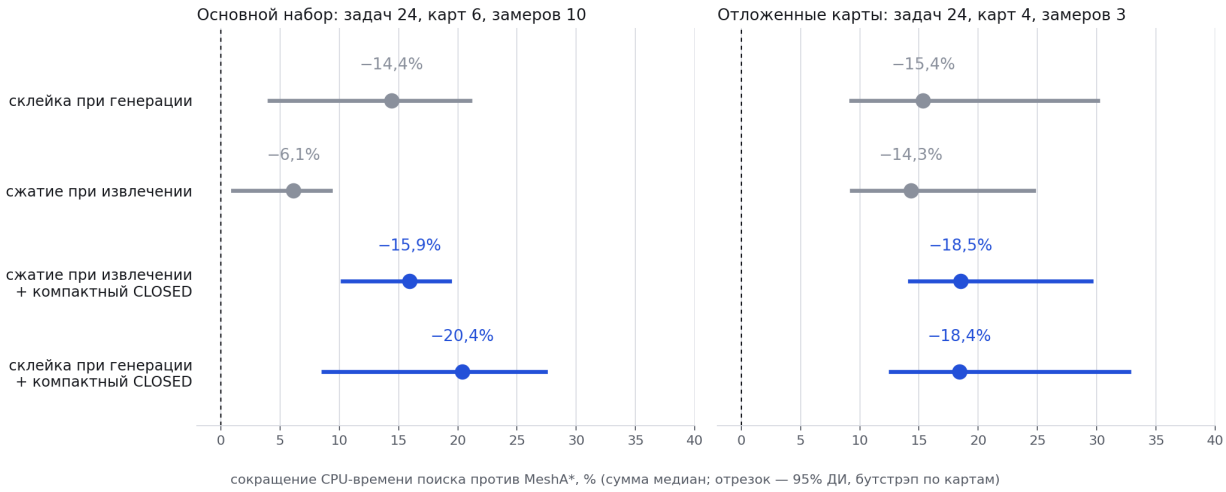
Теорема. Поиск, в котором цепочка заменена одним ребром той же стоимости, а все её клетки проверены на столкновения, — это A* на графе с теми же путями и стоимостями. Согласованность h сохраняется: $h(u) \le h(w_1) \le \cdots \le h(e) \le c(e,x) + h(x)$, цель всегда initial и не может оказаться внутри цепочки. Следовательно, поиск полон и оптимален при согласованной эвристике. Условия лемм проверяет `MacroPaths::build`; скрипт `verify_tree.py` подтверждает их на всех таблицах: 864 и 368 internal-конфигураций, 364 и 204 звена цепочек.

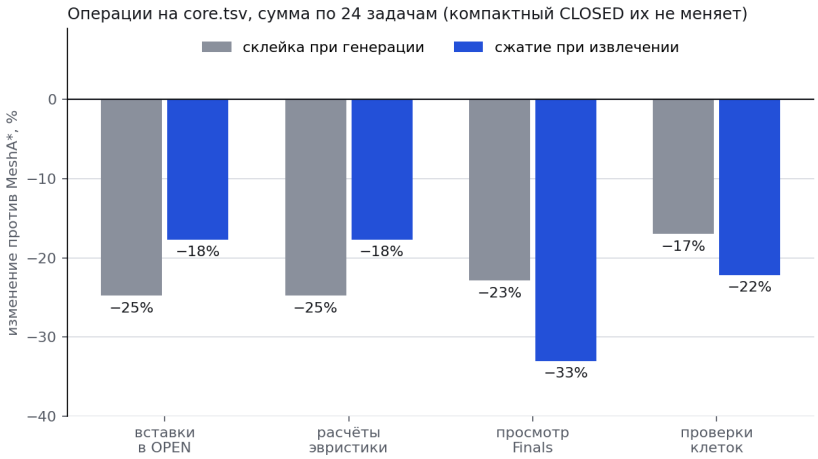
Следствие (компактный CLOSED). Internal-клетки не повторяются, поэтому хранить их в CLOSED не нужно. Флаг `COMPACT_CLOSED` оставляет в CLOSED только initial-клетки со слоем θ: 16 слоёв вместо 1800, при карте 1200×1200 — 2,7 МиБ вместо 309 МиБ. Режим включается только если `MacroPaths` подтвердил структуру дерева. В корневом CLI он включён по умолчанию (`make COMPACT_CLOSED=0` — выключить).

Сравнение на Linux. Скрипт `run_compare.py` запускает пять вариантов из одних исходников: исходный MeshA*, склейку при генерации, склейку с компактным CLOSED, сжатие цепочки при извлечении вершины (альтернативная реализация, `GLUING_MODE=4`) и его вариант с компактным CLOSED. Основной набор — два сеанса по пять замеров с обратным порядком вариантов, отложенные карты — три замера. Каждый процесс однопоточный и закреплён за своим CPU через `sched_setaffinity`, одновременно работали два процесса на разных CPU. Машина: Linux-ВМ, 4 виртуальных CPU.

Вариант	Основной	Отложенные	Быстрее	Расхождений
Склейка при генерации	−14,4%	−15,4%	23/24	0
Склейка + компактный CLOSED	−20,4%	−18,4%	24/24	0
Сжатие при извлечении	−6,1%	−14,3%	22/24	0
Сжатие при извлечении + компактный CLOSED	−15,9%	−18,5%	24/24	0

Лучший вариант — склейка с компактным CLOSED: на основном наборе CPU-время меньше на 20,4% (95%-интервал ускорения 1,09–1,38 ×), на отложенных картах — на 18,4%. Склейка без компактного CLOSED дала 14,4% на основном наборе, что согласуется с серией на M1 (16,4%).





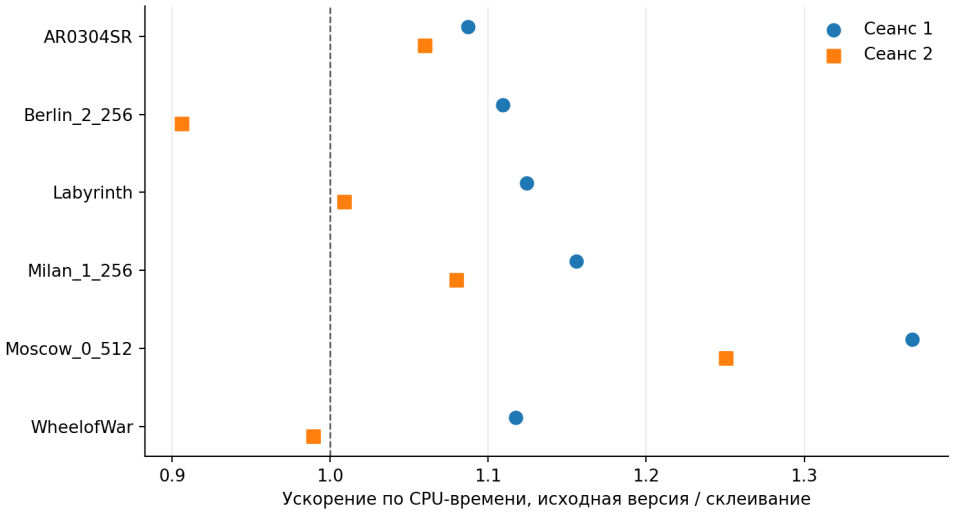
Склейка при генерации кладёт в OPEN одну вершину на цепочку, поэтому сильнее сокращает вставки и расчёты эвристики. Сжатие при извлечении сильнее сокращает просмотр Finals, но кладёт в очередь потомков конца цепочки. Компактный CLOSED не меняет число операций: выигрыш даёт маска, которая помещается в кэш процессора. Число удалённых дубликатов (initial-клетки) растёт, но дубликатов internal-клеток нет ни в одном варианте.

Ограничения: на отложенных картах только три замера; одновременно работали два процесса, поэтому возможна взаимная нагрузка на память; результаты относятся к этой машине и этим задачам.

Повторяемость и запуск

Измерения выполнены на Apple M1 с восемью ядрами. У программы один вычислительный поток. Основное время получено через `CLOCK_THREAD_CPUTIME_ID`, обычное время записано через `steady_clock`. Это убирает из основной метрики паузы вытеснения потока, но не влияние частоты CPU и кэшей.

Жёсткой привязки к физическому ядру в этом запуске нет. Вызов `THREAD_AFFINITY_POLICY` вернул `KERN_NOT_SUPPORTED (46)`. Одинаковый `QOS_CLASS_USER_INITIATED` задаёт предпочтение планировщику, а не закрепление. Это соответствует назначению affinity API macOS. Нагрузка во втором сеансе была выше. Медиана относительного MAD CPU-времени по задачам составила 2,4% для исходной версии и 5,0% для склейки.



Код также поддерживает `sched_setaffinity` в Linux. Там выбирается разрешённое ядро и проверяется результат закрепления. Linux-прогон основной серии ABBA/BAAB здесь не выполнялся; сравнение вариантов на Linux приведено в дополнении. Проценты основной серии относятся к этому Mac и этим наборам задач.

В журнале основной серии сохранены команды, порядок, семена и хеши бинарников. Манифест данных и контрольные суммы исходников позволяют проверить входные файлы. Сырые CSV сохранены для каждого процесса. Никакие неудобные повторы из основной серии не удалялись.

Команды из корня распакованного проекта:

```
bash research/gluing/build.sh
caffeinate -i python3 research/gluing/run_confirm.py
python3 research/gluing/analyze.py
python3 research/gluing/run_holdout.py
python3 research/gluing/profile.py
```

В Linux запускать `run_confirm.py` без `caffeinate`. Компилятор задаётся через `CXX`. Для измерений нужны Python 3 и компилятор C++17. Для графика нужен matplotlib.

CLI собирается скриптом `research/gluing/build_cli.sh`. Бинарник `mesh_astar_glued` принимает авторские аргументы. На примере с обходом стены стоимость осталась 74,4033, число раскрытий уменьшилось с 91 451 до 69 586. Разовые времена CLI в оценку ускорения не входят.

Патч Таблицы цепочек Изменения генерации потомков

Основные результаты Отложенные карты